AURA

FROM ZERO KNOWLEDGE TO A WORKING JAVA PROJECT

AURA Project

Complete Technical Learning Manual

A self-study course for understanding how this project was planned, designed, programmed, connected, tested, and finalized. It is written for a beginner who did not personally write the code and needs to learn the normal engineering process step by step.

Technology

Java 17 | Maven | Swing + FlatLaf | JDBC | PostgreSQL / Supabase | JUnit 5 | BCrypt | Google OAuth

Team learning tracks

Mohammed Nafih

Muhammed Rinshid

Nirmal

Rahandeep

Adil Rahman

Current project position: approximately 70% complete

Use the source files beside this manual as the laboratory exercises.

How To Use This Manual

A guided study path for a team member or examiner new to AURA

Purpose

This manual explains how the AURA project was built, how the code is connected, where each feature lives, how the database protects the workflow, and how to reproduce the development process. Read it in order once, then use the member chapters and code maps as revision notes before the presentation or viva.

Recommended study sequence

1. **Understand the problem**

   AURA converts campus complaints and suggestions into anonymous, trackable work items.

2. **Learn the layers**

   UI starts the action, services apply rules, DAOs execute SQL, and PostgreSQL stores the result.

3. **Trace one workflow**

   Follow login, student submission, receipt creation, admin status update, and history recording.

4. **Open the source**

   Use the file/class map to locate each method described in the manual.

5. **Rehearse ownership**

   Each member page contains their responsibility, evidence to show, and a speaking script.

6. **Verify the build**

   Run the Maven tests and, when configured, run the application against the PostgreSQL database.

> **Important source note**
>
> The repository implements Java Swing with FlatLaf, not JavaFX. This manual describes the implementation that exists in the source tree. Architecture and roadmap documents may describe future or approved components; those are marked as planned when they are not present in code.

Step 1 | Set Up The Project

What a new developer does before changing code

Required environment

- Java: Java 17 is configured in pom.xml as both compiler source and target.
- Maven: Maven resolves PostgreSQL JDBC, FlatLaf, BCrypt, Gson, and JUnit Jupiter.
- Database: A PostgreSQL/Supabase database must contain sql/schema.sql and optionally sql/seed.sql.
- Configuration: Copy db.properties.example and oauth.properties.example into local properties files; keep secrets out of Git.
- Test isolation: src/test/resources contains the test configuration example. Tests that use the database require configured connection values.

Commands

Build and test

```
mvn test
# compile production code only
mvn -DskipTests package
# run the desktop entry point
mvn exec:java -Dexec.mainClass=aura.Main
```

First inspection order

Open pom.xml to learn dependencies, Main.java to find the entry point, LoginFrame.java to see the first screen, AuthService.java to see authentication rules, DatabaseSubmissionService.java to see the service implementation, SubmissionDAO.java to see SQL, and schema.sql to see the database contract.

Expected result

A successful setup compiles Java 17 sources, discovers the six JUnit test classes, and can start the Swing login window when local configuration is available.

Step 2 | Read The Project Layers

The dependency direction that keeps the system explainable

The AURA request path

User action

LoginFrame, StudentDashboardFrame, AdminDashboardFrame

Business decision

AuthService, HypeService, DatabaseSubmissionService

Persistence operation

UserDAO, SubmissionDAO, StudentReceiptDAO, SubmissionHypeDAO

Stored state

users, submissions, student_submission_receipts, submission_hype, resolution_notes, submission_history

How to decide where new code belongs

- UI: Only collect input, show output, respond to clicks, and route to a service.
- Service: Validate roles, coordinate multiple DAO calls, enforce business rules, and translate failures.
- DAO: Open a connection, use a parameterized SQL statement, map rows to models, and close resources.
- Model: Represent the data transferred between layers; do not perform database work.
- Config/util: Provide shared infrastructure such as connection configuration, OAuth properties, password hashing, validation, and session state.

Step 3 | Understand The Data Model

The objects that move through the application

Model class	Attributes	Why it exists
User	userId, name, email, passwordHash, role	Account and RBAC identity
Submission	submissionId, title, description, type, category, location, priority, status, photoUrl, hypeCount	Anonymous public ticket
ResolutionNote	noteId, submissionId, adminId, note, createdAt	Official admin response
SubmissionHistory	historyId, submissionId, oldStatus, newStatus, changedBy, changedAt	Lifecycle audit entry

Important design decision: identity is separated

Submission deliberately has no studentId attribute. The student-to-submission relationship is held by StudentReceiptDAO and the student_submission_receipts table. This allows a student to retrieve their own tickets while the administrative submission queue sees only anonymous ticket data.

SubmissionDAO creates the anonymous row and private receipt in one transaction

```
connection.setAutoCommit(false);
insertSubmission.executeUpdate();
submission.setSubmissionId(keys.getInt(1));
receipt.setInt(1, studentId);
receipt.setInt(2, submission.getSubmissionId());
connection.commit();
```

Viva answer

The receipt is not a display trick. It is a separate relational mapping, so the anonymity boundary survives even when the database is inspected directly.

Step 4 | Authentication From Click To Session

Detailed login implementation

Password login path

1. LoginFrame.passwordLogin reads the email field and password field.
2. AuthService.authenticate rejects null values and calls validateCredentials.
3. ValidationUtil checks the official @tkmce.ac.in pattern and an eight-character minimum password.
4. UserDAO.findByEmail uses a PreparedStatement against users.
5. PasswordUtil.verifyPassword compares the plaintext input with the BCrypt hash.
6. AuthService places the User in SessionContext when verification succeeds.
7. LoginFrame.openDashboard routes ADMIN to AdminDashboardFrame and other users to StudentDashboardFrame.

AuthService.authenticate

```
if (email == null || password == null) return null;
if (!validateCredentials(email, password)) return null;
User user = userDAO.findByEmail(email);
if (user == null || !PasswordUtil.verifyPassword(password,
        user.getPasswordHash())) return null;
SessionContext.setCurrentUser(user);
```

Google OAuth path

loginWithGoogle delegates browser authentication to GoogleOAuthService. The service starts a loopback HttpServer, creates state and PKCE values, opens the browser, receives the callback, exchanges the authorization code, retrieves verified profile data, and returns GoogleUserInfo. AuthService applies the institutional domain check, finds or provisions a STUDENT, and stores the session.

Local development

OAuthConfig supports a development mock mode. loginMock creates a demo STUDENT or ADMIN only when development mode is enabled; this is useful for UI work but is not a production authentication path.

Step 5 | Student Creates A Submission

The complete issue/suggestion workflow

1. **Open**

   After successful login, StudentDashboardFrame verifies the current session is a student and loads the personal list.

2. **Fill**

   SubmissionFormPanel collects title, description, type, category, location, priority, and optional photo URL.

3. **Validate**

   The panel requires the core fields before it constructs a Submission object.

4. **Call service**

   The form callback calls ServiceRegistry.getSubmissionService().createSubmission(submission, user.getUserId()).

5. **Write atomically**

   DatabaseSubmissionService delegates to SubmissionDAO.createWithReceipt, which inserts the anonymous submission and receipt together.

6. **Refresh**

   The dashboard reloads the student IDs from the receipt vault, retrieves matching Submission objects, and refreshes the JTable.

7. **Support**

   HypeService.addHype records a single student vote and the dashboard shows the updated count.

Service interface called by the UI

```
Submission createSubmission(Submission s, int studentId);
List<Submission> listSubmissionsByStudent(int studentId);
void updateStatus(int submissionId, String newStatus,
                  int changedBy);
```

What to demonstrate

Use the demo student, create an ISSUE, refresh the table, select it, add Hype, open details, and explain that the admin queue can see the ticket but not its author.

Step 6 | Admin Resolves A Submission

The operational lifecycle and audit trail

Admin workflow

- Guard: AdminDashboardFrame constructor rejects a session that is not authenticated as ADMIN.
- Load: ServiceRegistry returns the SubmissionService; listAllSubmissions loads the shared queue sorted by hype and recency.
- Select: The administrator chooses a row from the JTable and reads its submission ID.
- Transition: The Change Status control chooses PENDING, ASSIGNED, IN_PROGRESS, RESOLVED, or REJECTED.
- Record: DatabaseSubmissionService obtains the old status and asks SubmissionDAO.updateStatusWithHistory to update the ticket and insert the audit record in one transaction.
- Explain: Add Resolution Note writes the admin’s note through SubmissionDAO and the history view reads both notes and transitions.
- Logout: AuthService.logout clears SessionContext and returns to LoginFrame.

Transactional status update

```
connection.setAutoCommit(false);
update.setString(1, newStatus);
update.setInt(2, id);
update.executeUpdate();
history.setString(2, oldStatus);
history.setString(3, newStatus);
history.setInt(4, changedBy);
history.executeUpdate();
connection.commit();
```

Why history is separate

The submissions row answers ’what is the current state?’ The submission_history rows answer ’how did it reach this state and which administrator changed it?’ Keeping both supports a fast dashboard and an audit trail.

Failure behavior

If either the status update or history insert fails, the DAO rolls back the transaction and rethrows the SQLException. The ticket cannot silently change without its audit record.

Step 7 | Database Build Manual

How the relational contract supports the application

Run order

1. Create or connect to a PostgreSQL/Supabase project.
2. Run sql/schema.sql. It creates users first, submissions second, then dependent receipts, hypes, notes, and history.
3. Run sql/seed.sql for the presentation dataset: six users, five tickets, receipts, hypes, notes, and transitions.
4. Copy the database example properties into a local db.properties file and set db.url, db.user, db.password, and db.driver.
5. Start the application or test suite. DatabaseConfig loads the properties from the classpath and calls DriverManager.getConnection.

Table responsibilities

Table	Purpose
users	accounts, roles, BCrypt hashes, institutional email check
submissions	anonymous issue/suggestion content and current status
student_submission_receipts	private student ownership mapping
submission_hype	crowd support with duplicate-vote uniqueness
resolution_notes	official administrator response
submission_history	old/new state transitions and actor

Step 8 | JDBC And DAO Code

How Java turns an action into a safe SQL operation

Every DAO method follows this pattern

- 1. Prepare: Declare a SQL string with ? placeholders; never concatenate user input into SQL.
- 2. Connect: Use DatabaseConfig.getConnection() inside try-with-resources.
- 3. Bind: Set values using setInt, setString, or the appropriate PreparedStatement method.
- 4. Execute: Call executeQuery for reads or executeUpdate for writes.
- 5. Map: Convert ResultSet columns to a model object or collection.
- 6. Close: try-with-resources closes the connection, statement, and result set automatically.
- 7. Translate: The service layer catches SQL failures and exposes a runtime/application-level failure to the UI.

UserDAO.findByEmail

```
String sql = "SELECT user_id, name, email,"
        + " password_hash, role FROM users WHERE email = ?";
try (Connection connection = DatabaseConfig.getConnection();
     PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setString(1, ValidationUtil.normalizeEmail(email));
    try (ResultSet results = statement.executeQuery()) {
        return results.next() ? mapUser(results) : null;
    }
}
```

DAO ownership map

UserDAO owns user CRUD. SubmissionDAO owns submissions plus the compound receipt transaction, status history, and resolution notes. StudentReceiptDAO owns vault lookups. SubmissionHypeDAO owns vote operations. No UI class should import java.sql.* or call these classes directly.

Examiner checkpoint

PreparedStatement protects against SQL injection; foreign keys protect relationships; unique constraints protect one receipt and one hype per student-ticket pair.

Step 9 | Security And Privacy

Controls implemented in code and SQL

Threat / rule	Implementation	What it guarantees
Domain gate	ValidationUtil	Regex accepts only @tkmce.ac.in; schema adds CHECK constraint
Password protection	PasswordUtil + UserDAO	BCrypt hash is persisted; plaintext is not stored
SQL injection	All DAO classes	PreparedStatement placeholders and bound parameters
OAuth callback	GoogleOAuthService	Loopback server, state comparison, PKCE challenge, verified email
Session control	SessionContext + dashboard constructors	Current user and role are checked before screen access
Anonymity	SubmissionDAO + StudentReceiptDAO + schema	No student_id in submissions; ownership exists only in vault

The anonymity proof

An admin listing submissions receives Submission objects with title, description, category, location, priority, status, photo URL, and hype count. There is no student ID field to display. A student retrieves personal tickets by asking StudentReceiptDAO for submission IDs using the authenticated student ID, then loading the public submission records.

ValidationUtil institutional email rule

```
private static final Pattern TKMCE_EMAIL =
    Pattern.compile("^[A-Z0-9._%+-]+@tkmce\\.ac\\.in$",
                    Pattern.CASE_INSENSITIVE);
```

Security limitation to discuss honestly

The repository contains strong design and regression coverage, but full database integration and production OAuth verification still depend on local credentials and environment configuration.

Step 10 | Testing Manual

How the project checks behavior instead of relying only on the demo

Test class	Current suite	What to learn from it
AuthServiceTest	Three tests	Credential validation and authentication behavior
DatabaseSubmissionServiceTest	Three tests	Submission creation, listing, and service operations
HypeServiceTest	One test	Hype service behavior
PasswordUtilTest	Two tests	BCrypt hashing and verification
ValidationUtilTest	Two tests	Institutional email and password rules
AdminAnonymityRegressionTest	One test	Admin path cannot expose student identity

Test workflow

- Run: mvn test from the repository root.
- Read: Surefire reports are written under target/surefire-reports.
- Diagnose: A failure may mean a code defect, missing database properties, or an environment mismatch.
- Extend: For remaining work, add tests before calling report, analytics, notification, and UI improvements complete.
- Defend: Explain the behavior under test, the input, the expected output, and which layer owns the rule.

Representative JUnit shape

```
@Test
void invalidEmailIsRejected() {
    assertFalse(ValidationUtil.isValidTkmceEmail(
        "person@gmail.com"));
}
```

Class Map | Configuration And Utilities

Where to open the code when studying infrastructure

File	Package	Study explanation
Main.java	aura	Applies FlatDarkLaf and opens LoginFrame on the Swing event thread.
DatabaseConfig.java	aura.config	Loads db.properties/environment values, loads the driver, returns JDBC connections.
OAuthConfig.java	aura.config	Loads OAuth settings, enforced domain, server port, and development mode.
PasswordUtil.java	aura.util	Hashes and verifies passwords with BCrypt work factor 12.
ValidationUtil.java	aura.util	Checks institutional email, normalizes it, and checks password length.
SessionContext.java	aura.util	Stores the active User in a volatile process-local field and provides role checks.

Objects created during startup and login

- Main: creates LoginFrame through SwingUtilities.invokeLater
- LoginFrame: creates AuthService, text fields, password field, and buttons
- AuthService: creates GoogleOAuthService and UserDAO through its default constructor
- Successful login: creates or retrieves User and writes it to SessionContext
- Dashboard: creates ServiceRegistry-backed submission service and UI controls

Class Map | DAO And Models

The persistence objects and their relationships

Class	Package	Responsibilities and relationships
UserDAO	aura.dao	save, findById, findByEmail, update, delete; maps users table rows to User.
SubmissionDAO	aura.dao	save, findById, findByIds, findAll, updateStatus, createWithReceipt, history, notes, delete.
StudentReceiptDAO	aura.dao	Creates and retrieves private receipt mappings.
SubmissionHypeDAO	aura.dao	Adds/removes hypes and answers count/has-hyped queries.
User	aura.model	Identity and role fields; helper methods isAdmin/isStudent.
Submission	aura.model	Ticket fields; no author field by design.
ResolutionNote	aura.model	Administrative note data.
SubmissionHistory	aura.model	Status transition data and actor ID.

Call relationship

AuthService -> UserDAO -> User. DatabaseSubmissionService -> SubmissionDAO -> Submission, ResolutionNote, SubmissionHistory. StudentReceiptDAO -> submission IDs -> SubmissionDAO. HypeService -> SubmissionHypeDAO -> submission_hype.

Study tactic

For each DAO method, identify the SQL statement, input parameters, returned object, table touched, and whether a transaction is required. That five-question pattern covers most of the viva.

Class Map | Services And UI

The classes that turn actions into workflows

Class	Purpose	Called from
AuthService	Authentication, registration, Google login, mock login, logout, current-user checks	LoginFrame
GoogleOAuthService	Authorization code + loopback callback + PKCE + profile retrieval	AuthService
DatabaseSubmissionService	Concrete SubmissionService; wraps DAO calls and converts failures	ServiceRegistry, both dashboards
HypeService	Hype actions and duplicate handling	StudentDashboardFrame
ServiceRegistry	Provides the shared submission service	StudentDashboardFrame, AdminDashboardFrame
LoginFrame	Collects credentials and routes roles	Main
StudentDashboardFrame	Personal submissions, new form, hype, details, logout	LoginFrame
AdminDashboardFrame	All submissions, status, notes, history, logout	LoginFrame
SubmissionFormPanel	Collects and validates submission fields	StudentDashboardFrame

Why ServiceRegistry exists

The UI should not construct a new collection of DAOs or duplicate business rules. ServiceRegistry centralizes the concrete DatabaseSubmissionService used by both dashboards. It is a small composition point, not a second business layer.

Member Study Sheet | Mohammed Nafih

Detailed ownership, source files, and presentation preparation

RBAC, anonymity vault, and security pipeline

Main assignment	RBAC, anonymity vault, and security pipeline
Files to study	AuthService.java, GoogleOAuthService.java, PasswordUtil.java, ValidationUtil.java, SessionContext.java, StudentReceiptDAO.java
Speaking responsibility	Explain institutional login, BCrypt, OAuth state/PKCE, role-based access, and why the receipt vault protects anonymity.
Remaining work	Finish history edge cases, OAuth/database integration checks, documentation, and final integration review.

Study checklist

- [ ] Open every listed file and identify its package.
- [ ] Trace at least one method call from UI to database.
- [ ] Explain one validation or security rule.
- [ ] Demonstrate one feature using seeded data.
- [ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Muhammed Rinshid

Detailed ownership, source files, and presentation preparation

Team lead: system coordination, core architecture, and Supabase integration

Main assignment	Team lead: system coordination, core architecture, and Supabase integration
Files to study	Main.java, DatabaseConfig.java, LoginFrame.java, pom.xml, db.properties.example, architecture and integration documentation
Speaking responsibility	Explain the project goal, package boundaries, technology choices, startup path, Supabase connection, and how the four layers are integrated.
Remaining work	Coordinate final integration, configuration verification, end-to-end walkthrough, and release readiness.

Study checklist

[ ] Open every listed file and identify its package.

[ ] Trace at least one method call from UI to database.

[ ] Explain one validation or security rule.

[ ] Demonstrate one feature using seeded data.

[ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Nirmal

Detailed ownership, source files, and presentation preparation

Proposed solution, models, UML, and hype/trending engine

Main assignment	Proposed solution, models, UML, and hype/trending engine
Files to study	aura.model.* (User, Submission, ResolutionNote, SubmissionHistory), SubmissionHypeDAO.java, HypeService.java, UML/PRD documentation
Speaking responsibility	Explain the domain objects, fields, relationships, enums/state values, and how hype counts create a trending order.
Remaining work	Complete model cleanup, add stronger state/type abstractions where approved, and extend hype/trending tests.

Study checklist

[ ] Open every listed file and identify its package.

[ ] Trace at least one method call from UI to database.

[ ] Explain one validation or security rule.

[ ] Demonstrate one feature using seeded data.

[ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Rahandeep

Detailed ownership, source files, and presentation preparation

Database engineering and FlatLaf desktop GUI

Main assignment	Database engineering and FlatLaf desktop GUI
Files to study	sql/schema.sql, sql/seed.sql, aura.ui.student.*, SubmissionDAO.java, UserDAO.java
Speaking responsibility	Defend the six-table schema, prepared statements, transactions, indexes, foreign keys, seeded demo records, and student interface workflow.
Remaining work	Expand DAO integration coverage, finish UI polish, and verify database failure paths against configured PostgreSQL.

Study checklist

[ ] Open every listed file and identify its package.

[ ] Trace at least one method call from UI to database.

[ ] Explain one validation or security rule.

[ ] Demonstrate one feature using seeded data.

[ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Adil Rahman

Detailed ownership, source files, and presentation preparation

Audit lifecycle, admin dashboard, and analytics

Main assignment	Audit lifecycle, admin dashboard, and analytics
Files to study	aura.ui.admin.*, DatabaseSubmissionService.java, SubmissionService.java, history/note workflow, Report/Tracking design documentation
Speaking responsibility	Explain the administrator queue, status lifecycle, resolution notes, audit history, metrics/reporting design, and current implementation boundary.
Remaining work	Implement report generation, analytics, richer admin filters, notification decisions, and related tests.

Study checklist

- [ ] Open every listed file and identify its package.
- [ ] Trace at least one method call from UI to database.
- [ ] Explain one validation or security rule.
- [ ] Demonstrate one feature using seeded data.
- [ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Remaining Work | Step-By-Step Closure

How to turn the current 70% into a defensible final release

#	Workstream	Step-by-step action
1	Freeze current behavior	Run mvn test, record the baseline, and keep the working demo seed data.
2	Complete admin reporting	Add a report service/output, connect it to an admin control, and test the exported result.
3	Add analytics	Calculate counts by status/category/priority and expose them in a dashboard view.
4	Design notifications	Decide whether notifications are in-app, email, or status feedback; implement the chosen minimal path.
5	Polish UI	Replace placeholder layouts, add validation messages, loading/error states, and consistent sizing.
6	Expand tests	Cover DAO failure paths, status transition rules, receipt ownership, and integration configuration.
7	Run a bug sweep	Test invalid input, missing configuration, duplicate hype, unauthorized screens, and rollback behavior.
8	Update evidence	Refresh README, diagrams, screenshots, manual, and each member's presentation notes.

Definition of done for the next review

Every presented feature should have a visible path in the UI, a service method that owns its rule, a DAO/database operation where persistence is needed, a test or repeatable verification procedure, and an assigned member who can explain it line by line.

Quick Viva Revision

Questions an unfamiliar person should be able to answer

What is AURA?	A campus governance platform for anonymous issues and suggestions with student tracking and administrator resolution.
Why is the submission anonymous?	The submissions table and Submission model contain no student_id; identity is isolated in the student receipt vault.
Where does SQL live?	Only in aura.dao classes, using DatabaseConfig connections and PreparedStatement.
How does a student find their own tickets?	StudentReceiptDAO finds IDs for the authenticated student, then SubmissionDAO loads the anonymous submissions.
How is a password protected?	PasswordUtil hashes with BCrypt before UserDAO saves it and verifies it during authentication.
How is an administrator identified?	User.Role is stored as ADMIN, SessionContext exposes isAdmin, and AdminDashboardFrame checks it before opening.
How is a status change auditable?	SubmissionDAO updates submissions and inserts submission_history in one transaction.
What is complete?	Core authentication, submission, student tracking, hype, admin resolution, persistence, security controls, and baseline tests.
What remains?	Reporting, analytics, notifications, UI polish, wider tests, bug fixing, and documentation closure.
What should never be claimed?	Do not claim JavaFX, report generation, analytics, or production OAuth validation as complete unless those code paths are actually added and tested.

Build Process | How To Work Without An Agent

A normal beginner-friendly engineering workflow

- 1. Ask what problem exists: Write the problem in one sentence before creating a class. For AURA: students need a safe campus issue channel and administrators need an auditable work queue.
- 2. List users and actions: Identify Student and Admin, then list login, submit, view own tickets, hype, review, status update, resolution note, and logout.
- 3. Freeze requirements: Create a PRD for behavior and a TRD for technology, constraints, security, and database rules.
- 4. Design before coding: Draw the use-case, package/layer, class, sequence, and ER diagrams. Decide where each rule will live.
- 5. Create the database contract: Write tables, primary keys, foreign keys, checks, unique constraints, and indexes before DAOs.
- 6. Create plain models: Make Java objects that mirror the data without putting SQL inside them.
- 7. Build one vertical slice: Implement login from UI to service to DAO to database, test it, then repeat for submission and resolution.
- 8. Integrate by contracts: Members agree on class names, method signatures, table columns, and return values before combining branches.
- 9. Test every boundary: Unit-test rules, regression-test privacy, test invalid input, and run an end-to-end golden path.
- 10. Review and finish: Compare requirements to actual code, mark missing features honestly, fix bugs, document setup, and rehearse the explanation.

Build Process | Project Phases

What the team should have learned at each stage

Stage	Focus	What is learned	Evidence
Phase 1	Requirements	PRD/TRD, actors, use cases, security/privacy requirements	docs/PRD.md, docs/TRD.md
Phase 2	Architecture	Package boundaries, class responsibilities, UML, ER model	docs/ARCHITECTURE.md
Phase 3	Database + models	schema.sql, seed.sql, User, Submission, notes, history	sql/ and aura.model
Phase 4	DAO/JDBC	Connection configuration, prepared statements, row mapping, transactions	aura.config and aura.dao
Phase 5	Services	Business rules, session, submission orchestration, hype	aura.service and aura.util
Phase 6	Authentication	Password login, BCrypt, domain validation, OAuth, RBAC	AuthService, OAuth, LoginFrame
Phase 7	Workflows	Student create/track and admin review/resolve	student/admin UI and services
Phase 8	Verification	JUnit, anonymity regression, manual golden path, bug list	src/test and docs/TESTING.md
Phase 9	Finalization	Reports, analytics, notifications, UI polish, documentation	remaining 30% plan

The key lesson

Do not begin by asking an agent to generate every class. Begin with a requirement and a small testable slice. Every later class should exist because a requirement, relationship, or boundary requires it.

Build Process | How Members Connect Their Work

Integration contracts for a five-person team

Shared contracts

- Names: Agree that the source class and database table names are stable before implementation. Example: SubmissionDAO, student_submission_receipts, submission_history.
- Inputs: Document what each method receives: IDs, User, Submission, status strings, and note text.
- Outputs: Document whether it returns User, Submission, a List, a boolean, void, or throws an exception.
- Ownership: The UI calls services; services call DAOs; DAOs call JDBC. A member can work independently when this boundary is respected.
- Data contract: A model's field must match the SQL column that stores it and the DAO mapping that reads it.
- Error contract: Decide how invalid input, missing users, duplicate hype, missing configuration, and SQLException are represented.
- Integration test: After combining work, test the same golden path: login -> submit -> receipt -> admin update -> history -> logout.

Example handoff

Rahandeep's SubmissionDAO.createWithReceipt must produce the submission ID that the service and student UI need. Nafee's StudentReceiptDAO must query the same submission_id type. If the method names, IDs, or table columns disagree, integration fails even when each member's code compiles alone.

Normal collaboration artifacts

Use a task board, short design notes, a branch per feature, small commits, code review, and a shared test command. Agents can accelerate typing, but the team still has to own the design, verify the output, and be able to explain every line.

Build Process | Finalization

How the current 70% becomes a complete learning project

Work	Lead	Learning and implementation step
Reports	Adil Rahman	Design ReportService/output, connect admin action, test generated CSV.
Analytics	Adil Rahman + Muhammed Rinshid	Define metrics, query or aggregate data, display counts and trends.
Notifications	Adil Rahman	Choose in-app/status feedback or email; implement and test the decision.
Models/enums	Nirmal	Align documented UML with actual model classes and add approved typed enums if required.
Database/GUI	Rahandeep	Finish polished states, error messages, loading behavior, and integration coverage.
Security/privacy	Mohammed Nafih	Test OAuth/configuration, unauthorized paths, receipt ownership, and edge cases.
Integration/release	Muhammed Rinshid	Run full build, verify Supabase configuration, resolve merge issues, and maintain final walkthrough.
Team learning	All five	Each member teaches their track to another member and explains one dependency crossing into another track.

Final knowledge test

A project is learned when a team member can start from a requirement, identify the responsible class, explain the data flow, show the relevant code, run the feature, describe a failure case, and name the test that protects it.